

Final Report: *Smart Fruit Decay Monitoring System*

1. Introduction & Background

Fruits continue to release gases even after being harvested. As they ripen and decay, they emit **ethylene (C₂H₄)** and **methane (CH₄)**, gases that directly indicate freshness levels. High ethylene concentration speeds up ripening, and if left unchecked in storage areas, it can cause entire batches to spoil faster. But in this Experiment we not only focus on these two specific gasses but also on combinations of gasses which found during decay of fruits. Detecting these gases early is important to reduce **food waste** and **economic loss** in markets and storage facilities.

Traditionally, fruit quality is judged by appearance or smell, methods that are **manual, slow, and inaccurate**. This motivated the idea of building a **low-cost, automated system** that can continuously monitor gas emissions and environmental conditions around fruits. The aim was to detect early signs of decay using **gas sensors (MQ2 and MQ4)** along with **temperature, humidity, and pressure sensors (DHT22 and BMP280)**. A **DS3231 RTC** provided accurate timestamps, and a **GPS module** added location data for traceability.

In the **first phase**, the system used **Bluetooth Low Energy (BLE)** communication. An **Arduino Mega 2560** collected sensor readings, formatted them into structured records, and sent them through an **HM-10 BLE module** to an **ESP32 receiver**. The Mega also saved each record to an SD card for backup. To make communication reliable, a **custom protocol** with sequence numbers, CRC checking, and acknowledgment (ACK) was implemented, so no data was lost even if wireless interference occurred. The ESP32 acted as a BLE central device, verifying each record and forwarding it to a **local server** via HTTP POST requests for data logging and analysis.

This BLE version successfully achieved **real-time data transfer**, but required a constant power supply and stable connection. In closed environments like our **fruit decay experiment box**, the BLE signal sometimes weakened or dropped, leading to data gaps. To solve this, the **second phase** introduced **NFC (Near Field Communication)** for **offline data retrieval**. Here, the Arduino wrote the latest readings to an **NFC tag** using a PN532 module. Later, another Arduino with a PN532 reader could simply tap near the tag to read all data instantly, no continuous power or wireless link needed.

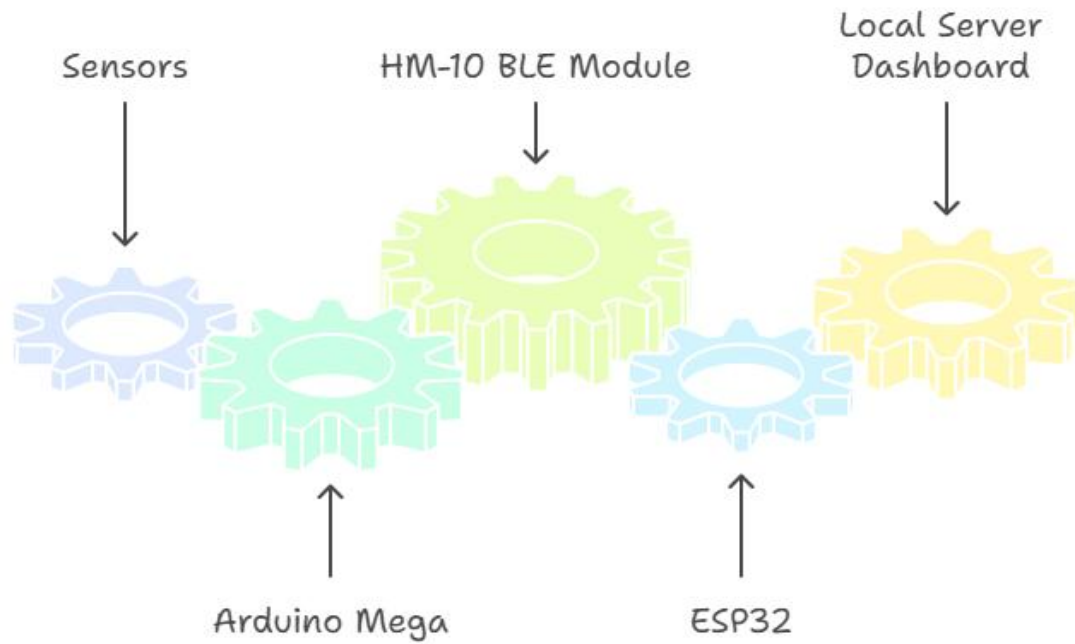
Finally, a sealed **banana decay box** was built to test the system. The sensors recorded MQ2 and MQ4 readings over time as the bananas ripened and spoiled. The data showed clear trends, gas levels rising during ripening and peaking before decay, proving the system could effectively track fruit freshness.

In summary, this project combines **sensor technology, embedded systems, and wireless communication** to create a smart, low-cost solution for early fruit decay detection, first using BLE for live monitoring, and later NFC for simple, power-efficient data access.

2. Phase 1: BLE-Based System

In the first development phase, the system was designed around **Bluetooth Low Energy (BLE)** communication to achieve **real-time wireless data transfer** between the sensing unit and a monitoring server. The setup used an **Arduino Mega 2560** as the **main data-acquisition and transmission node**, an **HM-10 BLE module** for short-range wireless transfer, and an **ESP32** as the **receiver and gateway** to the server.

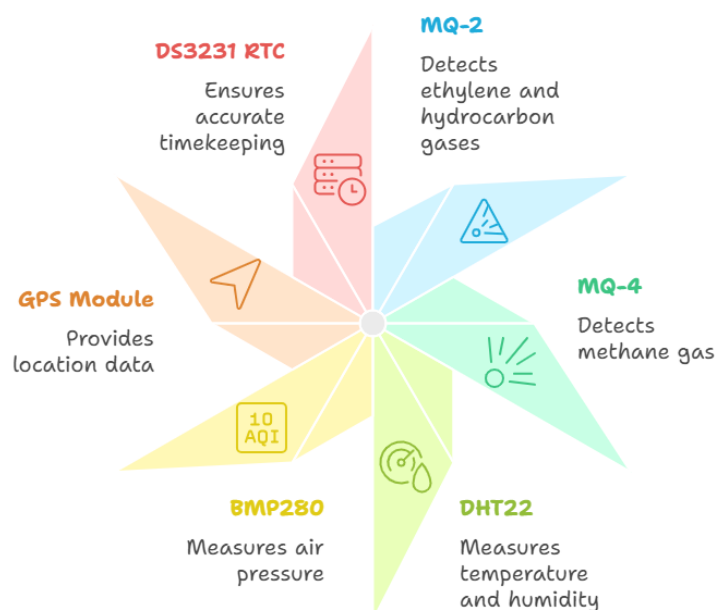
The overall flow was:



2.1 System Overview

The Arduino Mega continuously collects data from multiple sensors:

- **MQ-2** for ethylene or general hydrocarbon gases,
- **MQ-4** for methane,
- **DHT22** for temperature and humidity,
- **BMP280** for air pressure,
- **GPS module** for location,
- **DS3231 RTC** for accurate timekeeping.



Every interval (millisecond time interval is set by us, for example: 30 seconds, 1 minute, or 10 minutes.), the Mega reads all sensors, converts the analog voltages into meaningful physical values (ppm, °C, % RH, hPa), and tags the readings with a **precise UTC timestamp**. The readings are stored locally on an **SD card** in a **human-readable CSV file** (datalog.csv).

Once a record is ready, the Mega sends it over BLE through the HM-10 module to the ESP32. To make sure no packet is ever lost, a **custom reliable-transfer protocol** was implemented. Each data frame contains:

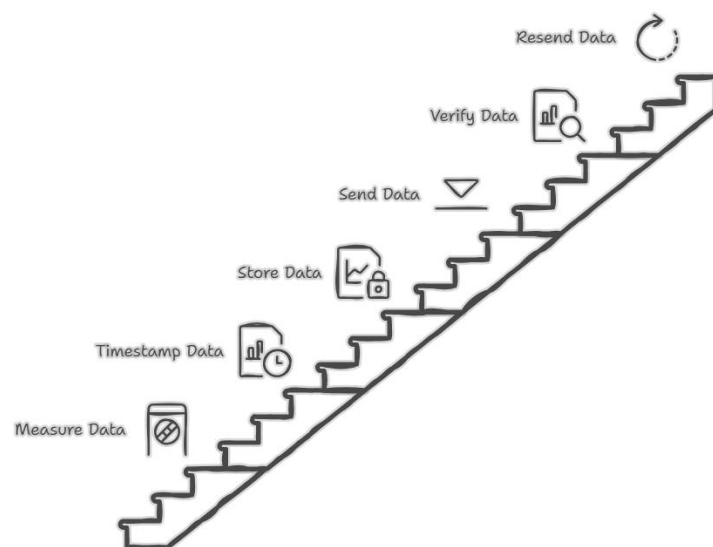
- a fixed header,
- length and sequence number,
- the data payload (sensor record),
- a **CRC checksum** for error detection.

The Arduino waits for an **ACK** (acknowledgment) from the ESP32 after sending each frame. If the ACK is not received within a short timeout, it automatically retries up to several times.

2.2 Sender Side (Arduino Mega + HM-10)

The sender acts as the brain of the system. It performs the following sequence:

1. **Measure:** Read all sensors (MQ-2, MQ-4, DHT22, BMP280, IR, GPS).
2. **Timestamp:** Get the exact time from the RTC; if GPS time becomes valid, it automatically corrects the RTC to true UTC.
3. **Store:** Save readings both as text (CSV) and as a binary record in SD card for backup.
4. **Transmit:** Package the same record, add header, sequence number, and CRC and send through HM-10.
5. **Confirm:** Wait for ACK from ESP32; if not received, retry.
6. **Sync on request:** When the ESP32 sends a “REQ” command, the Arduino streams all stored .bin files so any missed data can be recovered later.



2.3 Receiver Side (ESP32 BLE Central)

The ESP32 operates as the **central device** in BLE mode. It automatically scans nearby peripherals, detects the HM-10 module, and establishes a connection.

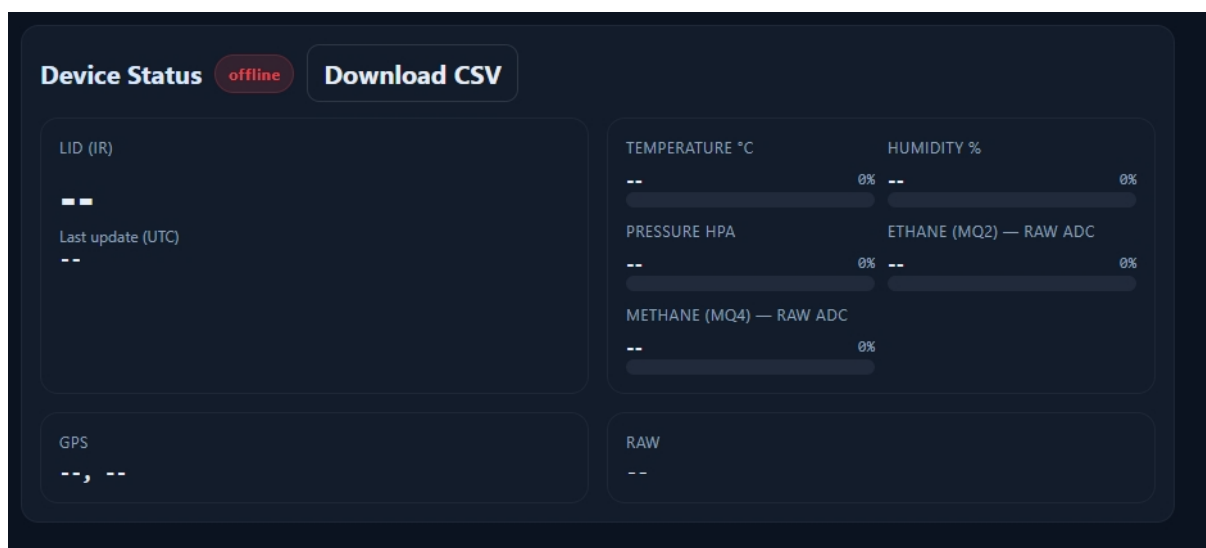
Once connected, it enables **notifications**, allowing the HM-10 to push data in real time.

Incoming data arrives in small 20-byte pieces. The ESP32:

1. **Reassembles** them into complete frames,
2. **Verifies** the CRC checksum to confirm integrity,
3. **Sends an ACK** containing the correct sequence number back to the sender,
4. **Parses** the payload into a structured record (timestamp, lat/lon, gas values, temperature, humidity, etc.),
5. **Publishes** it to a **local Flask web server** through an HTTP POST request in JSON format.

The web dashboard (like the one you designed—showing “Device Status,” “Temperature,” “Humidity,” “Gas Levels,” etc.) instantly updates whenever a new record arrives.

Here’s is our local server dashboard



If a packet fails CRC validation, the ESP32 ignores it and waits for the sender’s retry, maintaining data accuracy.

2.4 BLE Working Explanation

The complete BLE data cycle can be summarized as:

1. **Sensor Reading:** Arduino samples all connected sensors.
2. **Data Framing:** It packs the readings with timestamp, sequence, and CRC.
3. **Transmission:** The HM-10 sends the frame to the ESP32 in small BLE chunks.
4. **Acknowledgment:** The ESP32 checks CRC and replies with the same sequence number to confirm success.

5. **Logging & Display:** On receiving valid data, ESP32 posts it to the local server, which logs and visualizes it in real-time.

Three Liner Explanation:

The **Arduino Mega** collects and timestamps all sensor data, then reliably sends it through **HM-10 BLE** to an **ESP32**, which confirms every packet and uploads it to a **web dashboard**.

If any packet is lost, the system automatically resends it or allows the ESP32 to request old files.

3. Phase 2: NFC-Based System Integration

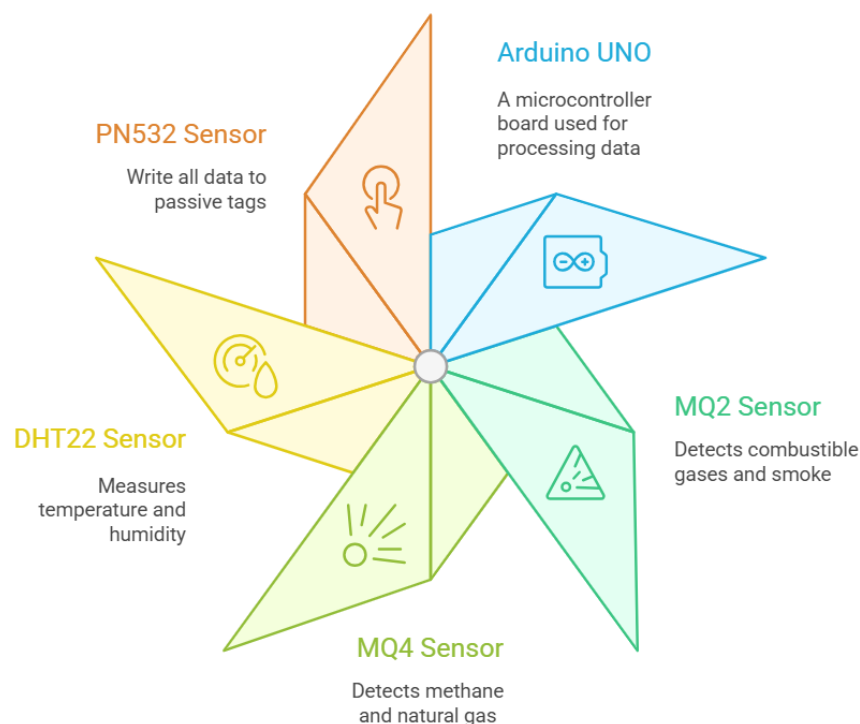
To make the system **low-power** and independent of any live (Wifi) wireless link, we integrated **NFC**. Unlike BLE (which needs two active radios), the NFC path lets the sensor node **sleep at microamp levels** and only wake long enough to write data when a tag is presented. This enables **offline data pickup**: a user taps a tag to **pull the latest and historical summaries** without any pairing, Wi-Fi.

3.1 Motivation

- **Power saving:** The PN532 (NFC module) stays in a light **standby (~10 μ A)** with RF (Radio Frequency) detector enabled. When a passive tag is brought near, it wakes, **writes at ~60 mA** for a brief moment, then goes back to standby. The rest of the node can also sleep between samples, so average current is tiny.
- **Offline data transfer:** No BLE link, no phone/ESP32 dependency. Just tap a tag to carry the data.
- **Simplicity & robustness:** No pairing, no signal issues; taps are deterministic and quick.

3.2 NFC Sender (Writer)

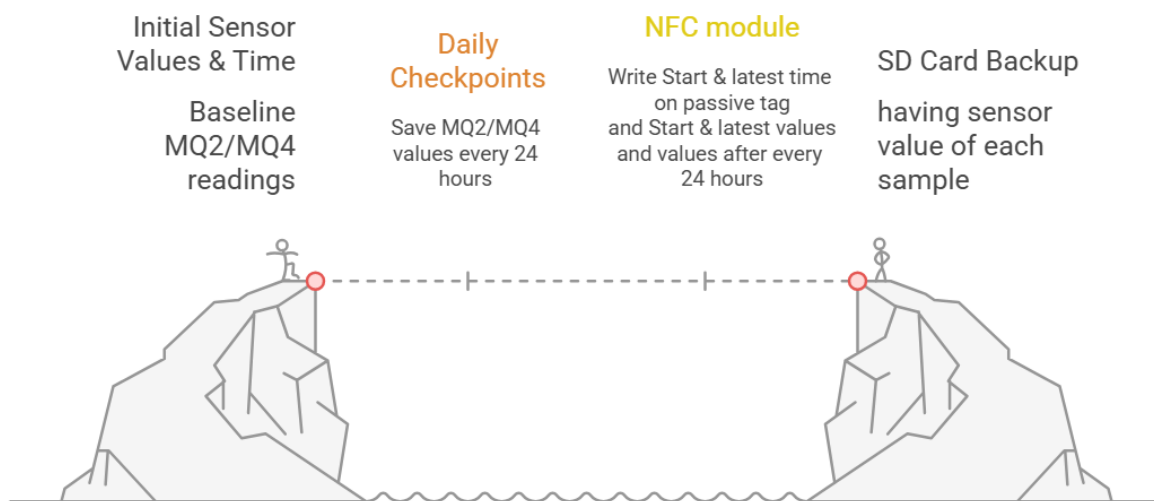
Hardware: Arduino UNO + MQ2, MQ4, DHT22, PN532.



Sampling: Period set by us e.g., **30 s / 1 min / 10 min** (or any ms interval).

What we record and store to the NFC passive tag (latest snapshot + dailies):

- **Start date/time** (when the experiment began).
- **Latest/Now date/time** (time of the most recent valid sample).
- **Starting MQ2/MQ4 values** (baseline at start).
- **Latest MQ2/MQ4 values** (most recent value).
- **Daily checkpoints:** MQ2/MQ4 values saved additionally **every 24 hours** labeled **Day 1, Day 2, ...** (to notice or just a quick view how much values changes).
- **SD card Integration:** Also append all the sensor values after every sampling in the SD card for backup.



How it works at runtime:

1. UNO samples MQ2/MQ4/DHT22 on the chosen interval and keeps the **current record** in RAM/EEPROM also append in SD card.
2. PN532 (nfc module) sits in **RF-detect standby**.
3. When **any passive MIFARE (a series of integrated circuit (IC) chips used in contactless smart cards) 1K tag** is presented, the writer:
 - Authenticates the target sector(s),
 - **Writes the current "latest snapshot"** (start/now times, start/latest MQ2/MQ4, and DHT22 values),
 - Updates/maintains the **per-day entries** (Day 1, Day 2, ...),
 - Returns to **standby**.

Tag memory model (MIFARE Classic 1K):

- **Total 1024 bytes = 64 blocks × 16 bytes.**
- We use **data blocks** only (e.g., **blocks 1–10** for the summary string; additional blocks for Day-entries).
- Write data on Passive tag like:
 - **Blocks 1–10:** Header & summary
 - Start: YYYY-MM-DD HH:MM:SS
 - Now: YYYY-MM-DD HH:MM:SS
 - MQ2_start / MQ4_start, MQ2_now / MQ4_now, Temp, Hum%
 - **Blocks 8–15: Daily checkpoints** (compact fixed-width fields like 1D:M2=x; M4=y\n, etc.) and then in last line all read data printed.

One LINER: the UNO measures; the PN532 only wakes to **burn the latest facts** into the tag when it “sniffs” a tag nearby then sleeps again.

3.3 NFC Reader (Receiver)

Hardware: A second Arduino + **PN532** + **1.3” I²C OLED**.

Behavior:

- When a tag is tapped, the reader authenticates and **reads the specific blocks** we use.
- It **reconstructs the full message** and prints **everything** on the **serial monitor** (including all per-day entries).
- Read data from Passive tag also in same manner and print on Serial Monitor:
 - **Blocks 1–10:** Header & summary
 - Start: YYYY-MM-DD HH:MM:SS
 - Now: YYYY-MM-DD HH:MM:SS
 - MQ2_start / MQ4_start, MQ2_now / MQ4_now.

```
Reading MIFARE Classic (slots layout)...
Block 1: 53 44 3A 32 30 32 35 2D 31 30 2D 32 38 20 20 20 | SD:2025-10-28
Block 2: 53 54 3A 31 33 3A 33 34 3A 35 30 20 20 20 20 20 | ST:13:34:50
Block 4: 4E 44 3A 32 30 32 35 2D 31 30 2D 33 31 20 20 20 | ND:2025-10-31
Block 5: 4E 54 3A 31 33 3A 33 34 3A 35 36 20 20 20 20 20 | NT:13:34:56
Block 6: 53 6D 32 3D 39 38 39 20 53 6D 34 3D 39 39 37 20 | Sm2=989 Sm4=997
Block 8: 4D 71 32 3D 32 20 4D 71 34 3D 36 35 32 20 20 20 | Mq2=2 Mq4=652
```

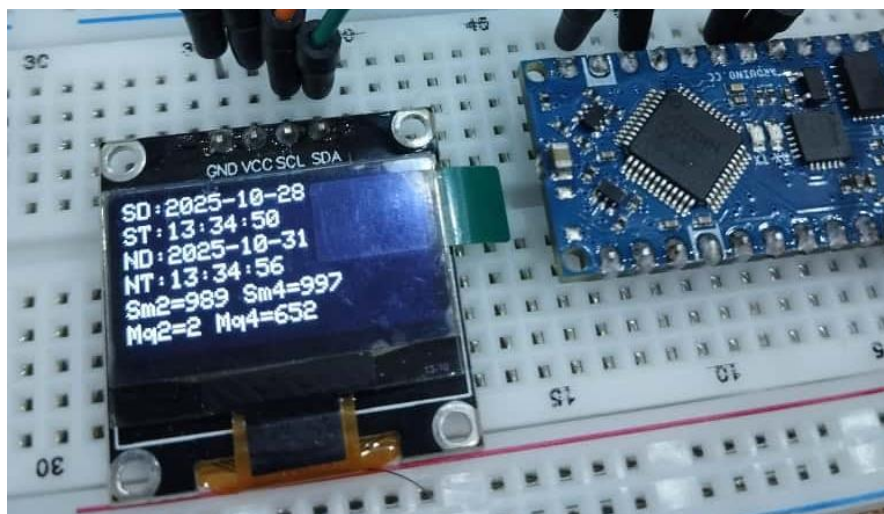
- **Blocks 8–15: Daily checkpoints** (compact fixed-width fields like 1D:M2=x; M4=y\n, etc.) and then in last line all read data printed.

```

Reading Classic daily snapshot blocks...
Block 9: 31 44 3D 32 36 30 36 30 2C 30 20 20 20 20 20 | 1D=26060,0
Block 10: 32 44 3D 32 35 39 2C 30 20 20 20 20 20 20 20 | 2D=259,0
Block 12: 33 44 3D 32 2C 36 35 32 20 20 20 20 20 20 20 | 3D=2,652
[TAG] start=2025-10-28 13:34:50 | now=2025-10-31 13:34:56 | Smq2=989 Smq4=997 | MQ2=2 | MQ4=652 | 1Dm2=26060 1Dm4=0 | 2Dm2=259 2Dm4=0 | 3Dm2=2 3Dm4=652
Blocks read: 9, Failed: 0

```

- On the **OLED** (limited space), it shows the **key summary only**:
 - **Start time & date**
 - **Now time & date**
 - **Start MQ2/MQ4**
 - **Latest MQ2/MQ4**



This gives a quick glance on the OLED, with the complete details available over serial.

3.4 NFC Working Explanation

End-to-end flow (write → read):

1. **Measure loop (UNO):** MQ2/MQ4/DHT22 sampled on schedule; “Start” locked at T0; “Now” updated each cycle; daily checkpoint stored every 24 h.
2. **Wait for tag:** PN532 idle with RF detector (~10 μ A).
3. **Tag presented:** PN532 wakes; writer **updates blocks** (summary + relevant daily entries) in a burst (~60 mA momentarily); back to standby.
4. **Read side:** Reader PN532 authenticates the same blocks, **reads & reconstructs** the string/data, **prints all on serial**, and **shows summary on OLED**.
5. **Repeat:** Tap any time to pull a fresh snapshot without ever turning on a radio.

BLE vs NFC (in this project):

- **Speed/live view:** BLE streams continuously to a server (good for dashboards). NFC gives snapshots on demand.

- **Power:** BLE keeps the radio alive on both ends; NFC tag is passive; writer's PN532 sleeps until a tap, **huge battery life win**.
- **Workflow:** BLE is "always-on monitoring"; NFC is "walk-up, tap, and take the latest + dailies".
- **Complexity:** BLE needs pairing and error-handling; NFC is simpler and deterministic.

4. Experimental Setup

4.1 Decay Box Experiment

To validate the Smart Fruit Decay Monitoring System, two complete rounds of laboratory experiments were carried out.

In each round, the sensing module **MQ-2 + MQ-4 + DHT22** was placed inside a small **sealed box** containing **ripe bananas** connected with Arduino. The box created a storage environment where gases released during ripening could accumulate naturally. Temperature and humidity were monitored to understand their influence on gas buildup.

Each measurement cycle was automatically taken at a **10-minute sampling interval**, controlled in code. The Arduino converted every analog reading to gas concentration using calibration formulas and stored time-stamped values. The main aim was to study how gas emission evolved as bananas ripened and decayed.

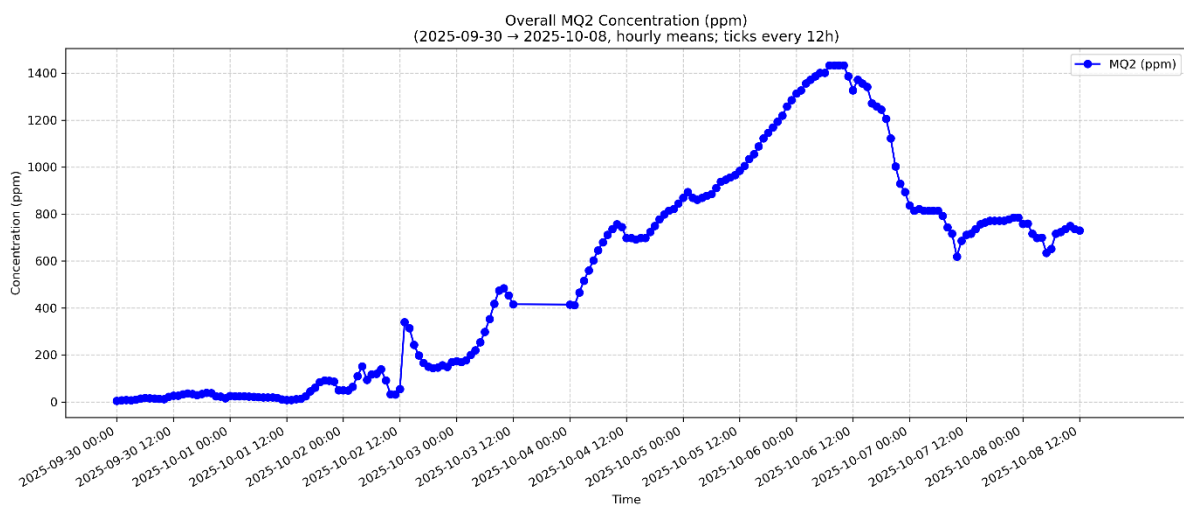
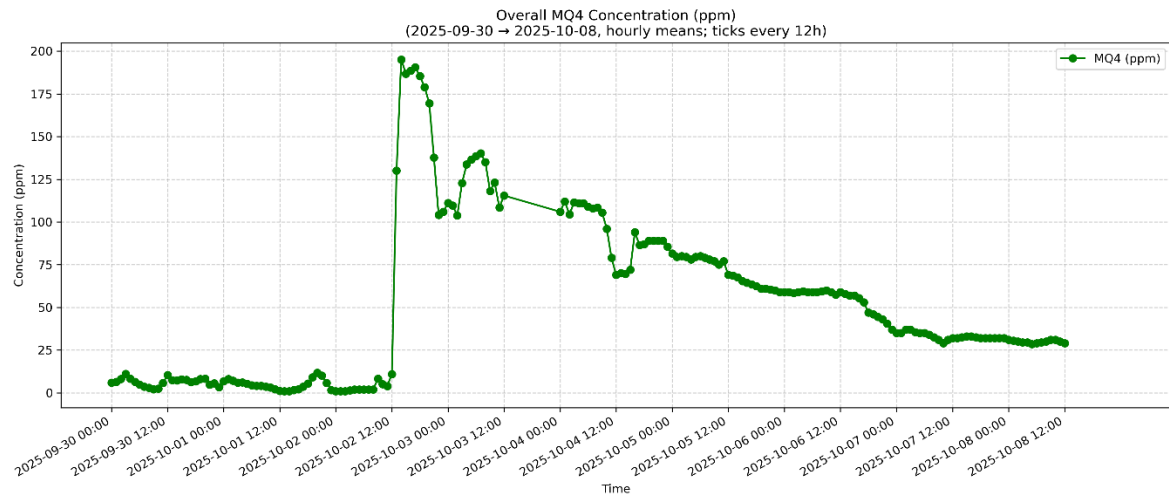
4.2 Data Collection Process

Each round contained **three distinct experiments**, repeated twice for consistency.

Round 1:

Experiment 1

- **Y-axis (PPM):** Parts per million; higher ppm = more target gas.
- **Pipeline:** $ADC \rightarrow V_{out} \rightarrow R_s = R_L \cdot (5 - V_{out}) / V_{out} \rightarrow (R_s / R_0) \rightarrow PPM$.
- **Terms:** **R_s** = sensor resistance (drops with more gas); **R_0** = clean-air baseline (from calibration); **R_L** = fixed load resistor.
- **Meaning:** **R_s / R_0** shows overall gas effect (mixture). **PPM** is a per-gas estimate using a fitted curve.
- **Curves used:**
 $MQ-2 \approx 50 \cdot (R_s / R_0)^{-2.3}$ (hydrocarbons/Ethylene);
 $MQ-4 \approx 1000 \cdot (R_s / R_0)^{-2.95}$ (Methane).
- **Reading the graph:** More gas $\rightarrow$ lower $R_s / R_0 \rightarrow$ higher ppm. **Peaks** = gas bursts; **flat/low** = clean or stable air.

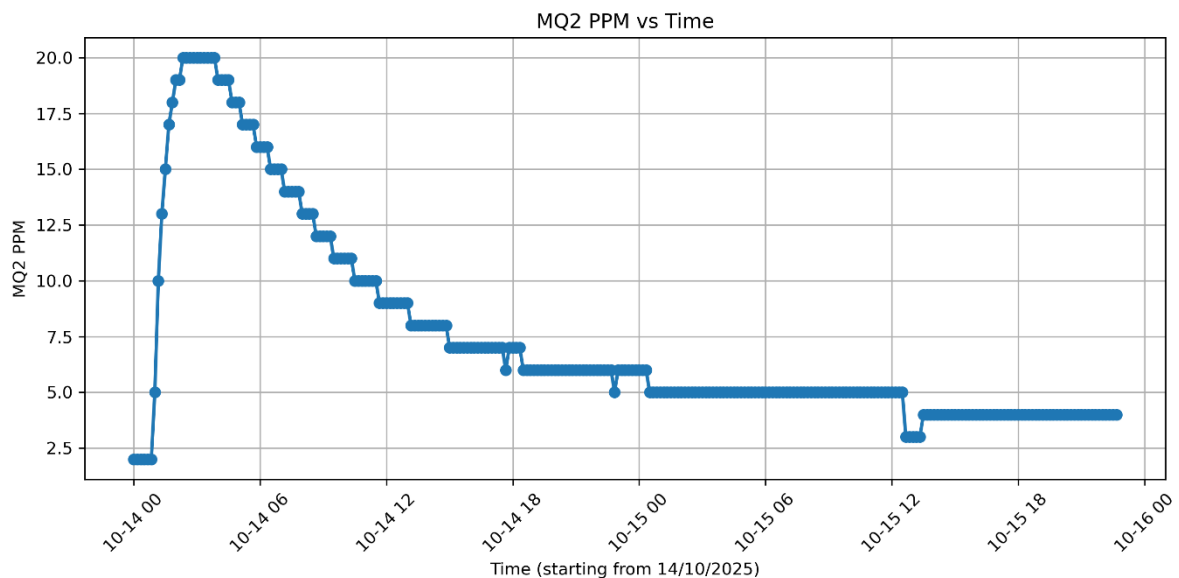
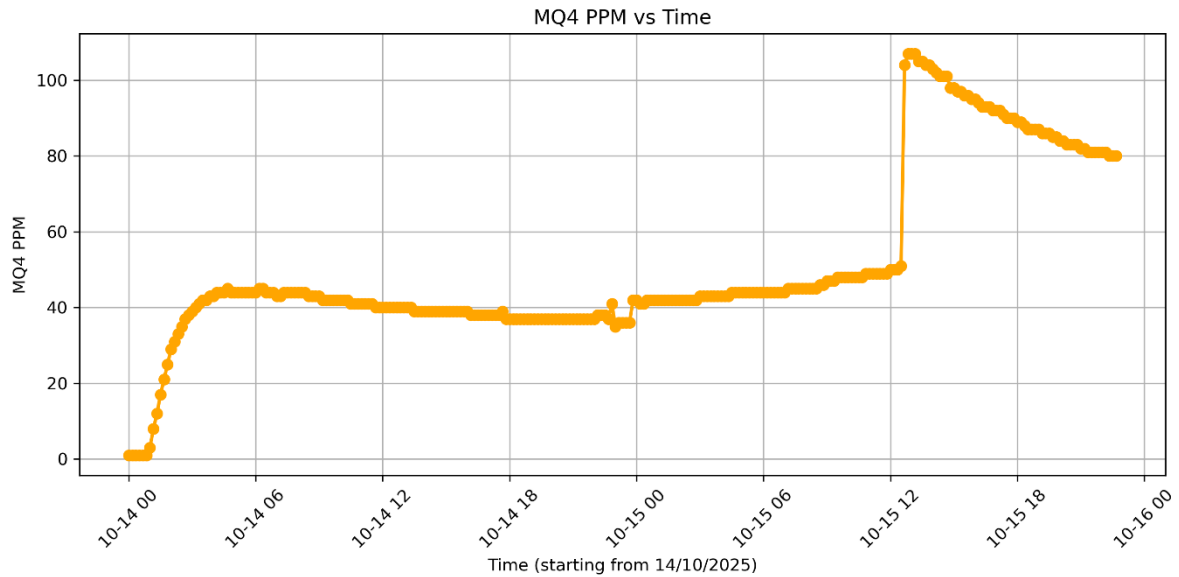


Experiment 2

- **Y-axis (PPM):** Parts per million; higher ppm = more target gas.
- **Pipeline:** $\text{ADC} \rightarrow \text{Vout} \rightarrow R_s = R_L \cdot (5 - \text{Vout}) / \text{Vout} \rightarrow (R_s / R_0) \rightarrow \text{PPM}$.
- **Terms:** R_s = sensor resistance (drops with more gas); R_0 = clean-air baseline (from calibration); R_L = fixed load resistor.
- **Meaning:** R_s / R_0 shows overall gas effect (mixture). **PPM** is a per-gas estimate using a fitted curve.
- **Curves used:**

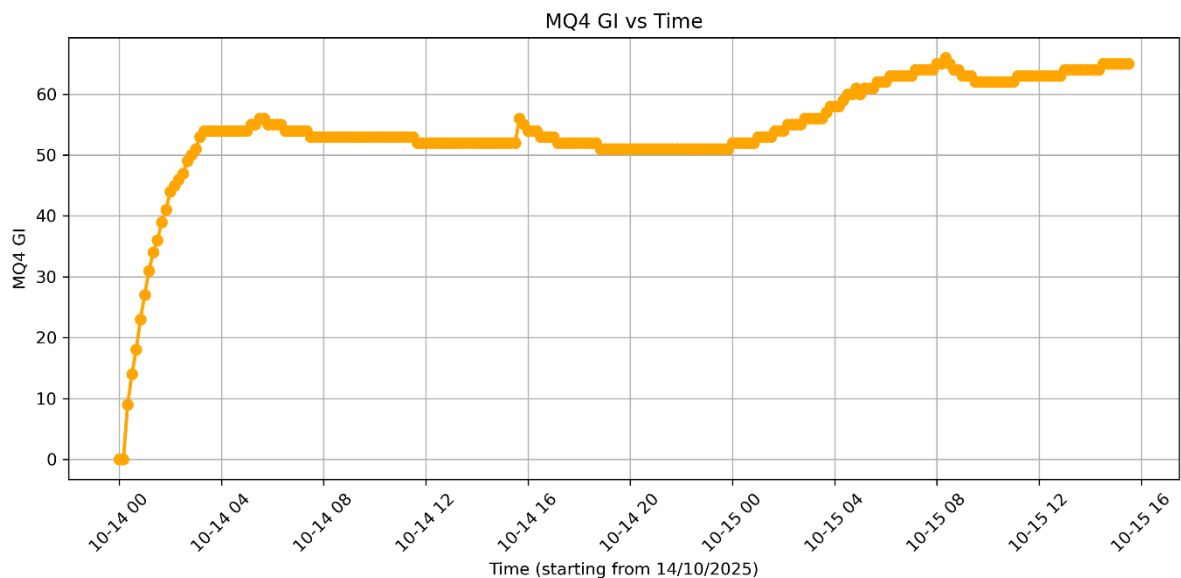
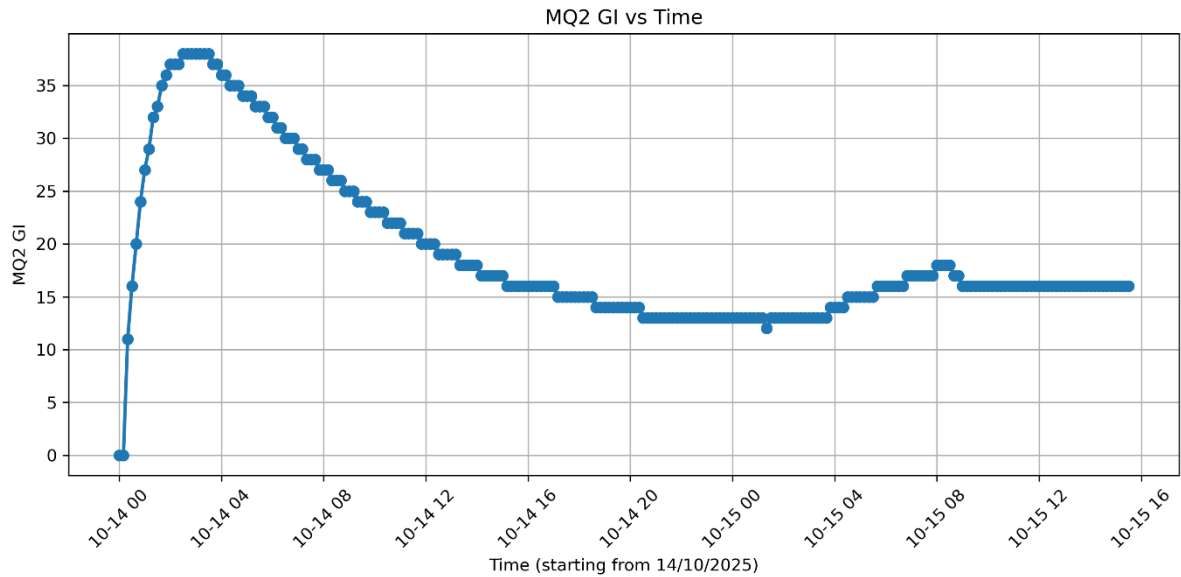
$\text{MQ-2} \approx 50 \cdot (R_s / R_0)^{-2.3}$ (hydrocarbons/Ethylene); $\text{MQ-4} \approx 1000 \cdot (R_s / R_0)^{-2.95}$ (Methane).

- **Reading the graph:** More gas $\rightarrow$ lower $R_s / R_0 \rightarrow$ higher ppm. **Peaks** = gas bursts; **flat/low** = clean or stable air.



Experiment 3 – Combined Gas Effect (Gas Index 0–100)

- **Y-axis** (Gas Index 0–100): Higher index = more target gas.
- **Pipeline:** ADC → Vout → $R_s = R_L \cdot (5/V_{out} - 1) \rightarrow (R_s/R_0) \rightarrow \text{GasIndex}$.
- **Terms:** R_s = sensor resistance (drops with more gas); R_0 = clean-air baseline (from calibration); R_L = fixed load resistor.
- **Meaning:** R_s/R_0 captures overall gas effect (mixture). GasIndex is a normalized, human-friendly scale (not gas-specific ppm).
- **Normalization mapping:**
 - $R_s/R_0 \geq 1.0 \rightarrow \text{GI} = 0$ (fresh air)
 - $R_s/R_0 \approx 0.6 \rightarrow \text{GI} \approx 40$
 - $R_s/R_0 \approx 0.3 \rightarrow \text{GI} \approx 80$
 - $R_s/R_0 \leq 0.1 \rightarrow \text{GI} = 100$

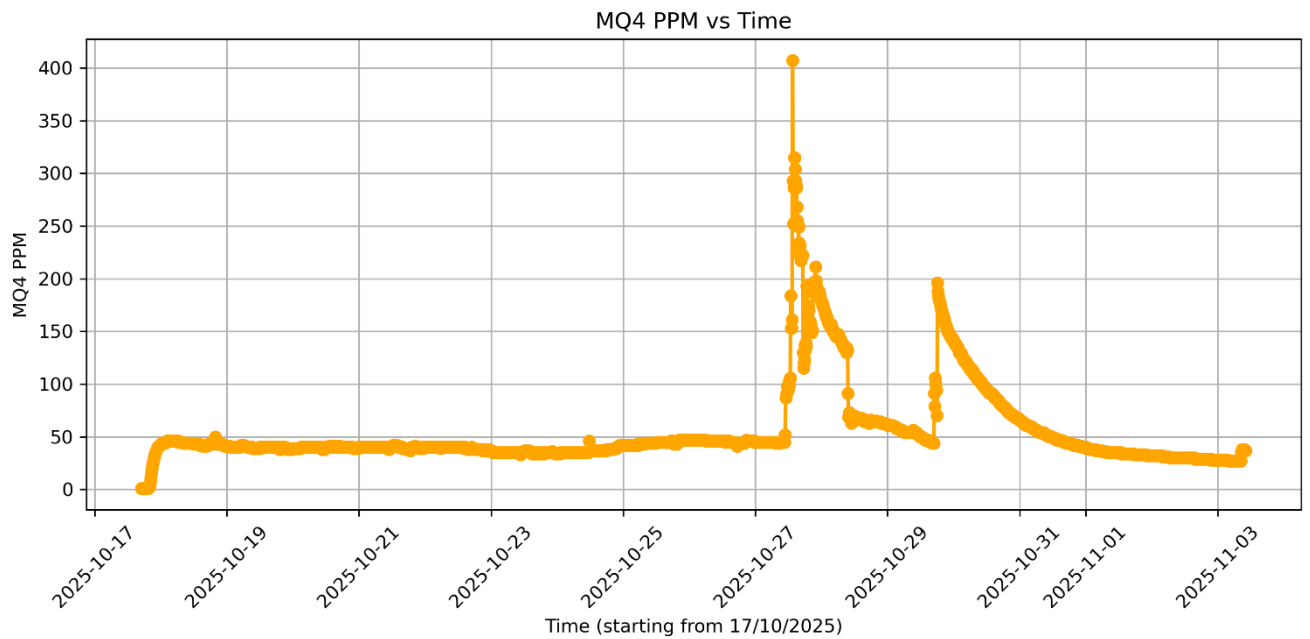
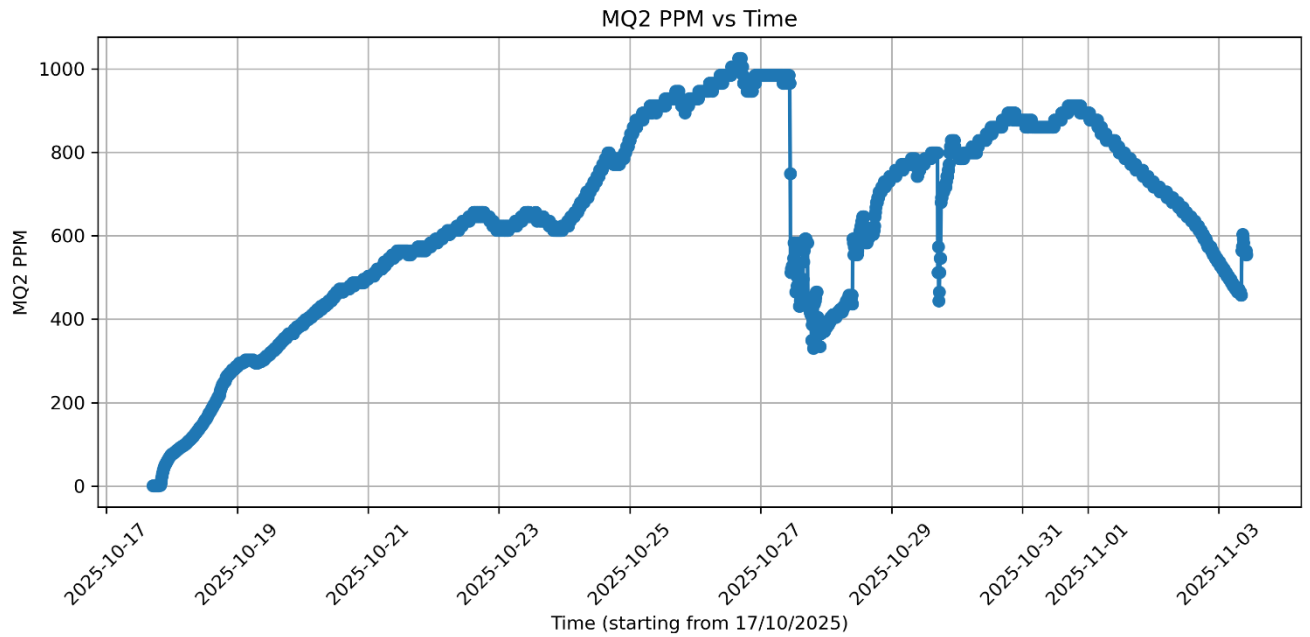


Round 2:

Experiment 1

- **Y-axis (PPM):** Parts per million; higher ppm = more target gas.
- **Pipeline:** $\text{ADC} \rightarrow V_{\text{out}} \rightarrow R_s = R_L \cdot (5 - V_{\text{out}}) / V_{\text{out}} \rightarrow (R_s / R_0) \rightarrow \text{PPM}$.
- **Terms:** R_s = sensor resistance (drops with more gas); R_0 = clean-air baseline (from calibration); R_L = fixed load resistor.
- **Meaning:** R_s / R_0 shows overall gas effect (mixture). **PPM** is a per-gas estimate using a fitted curve.
- **Curves used:**
 - $\text{MQ-2} \approx 50 \cdot (R_s / R_0)^{-2.3}$ (hydrocarbons/Ethylene);
 - $\text{MQ-4} \approx 1000 \cdot (R_s / R_0)^{-2.95}$ (Methane).

- **Reading the graph:** More gas → lower R_s/R_0 → higher ppm. **Peaks** = gas bursts; **flat/low** = clean or stable air.



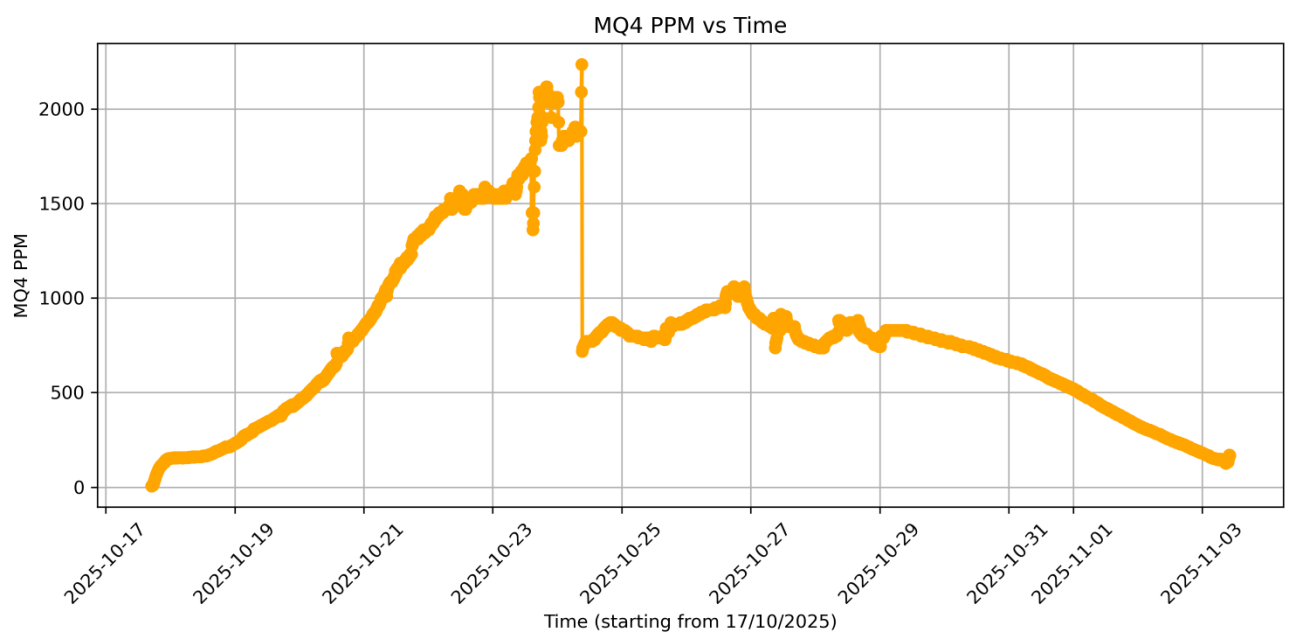
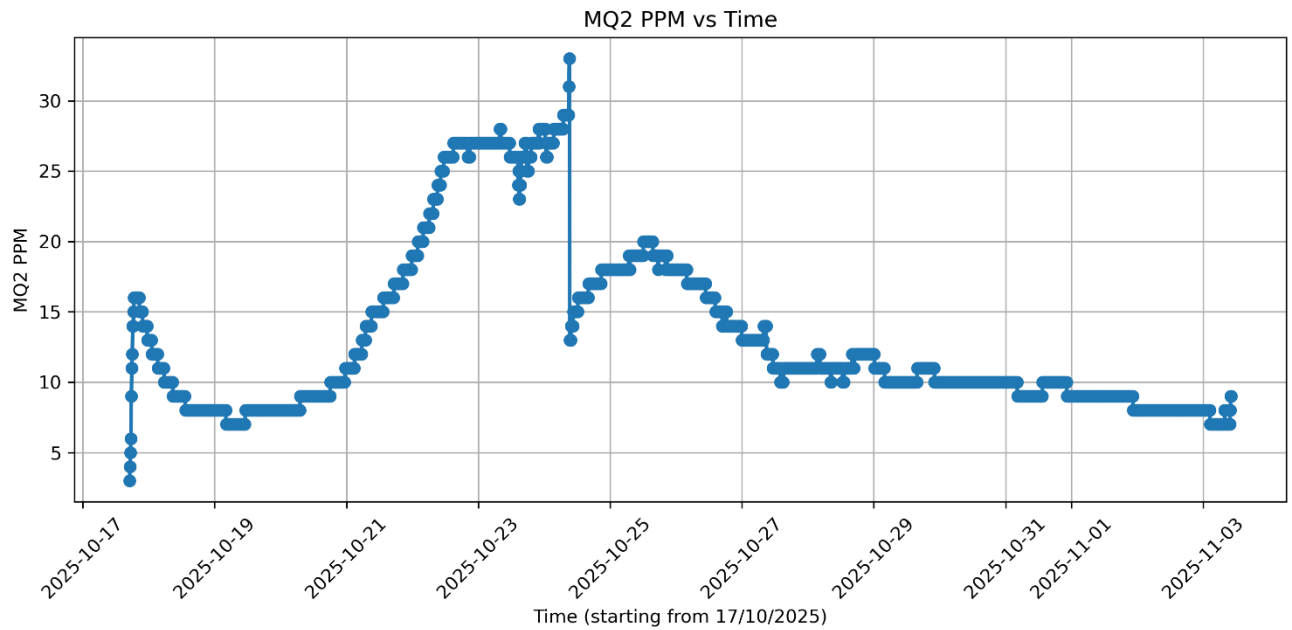
Experiment 2

- **Y-axis (PPM):** Parts per million; higher ppm = more target gas.
- **Pipeline:** $ADC \rightarrow V_{out} \rightarrow R_s = R_L \cdot (5 - V_{out}) / V_{out} \rightarrow (R_s/R_0) \rightarrow PPM$.
- **Terms:** R_s = sensor resistance (drops with more gas); R_0 = clean-air baseline (from calibration); R_L = fixed load resistor.
- **Meaning:** R_s/R_0 shows overall gas effect (mixture). **PPM** is a per-gas estimate using a fitted curve.

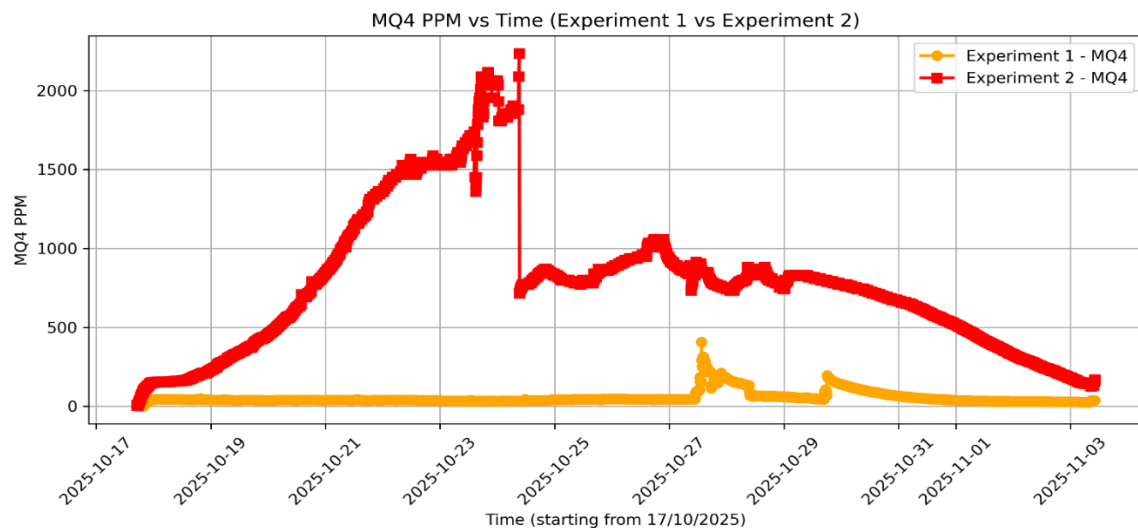
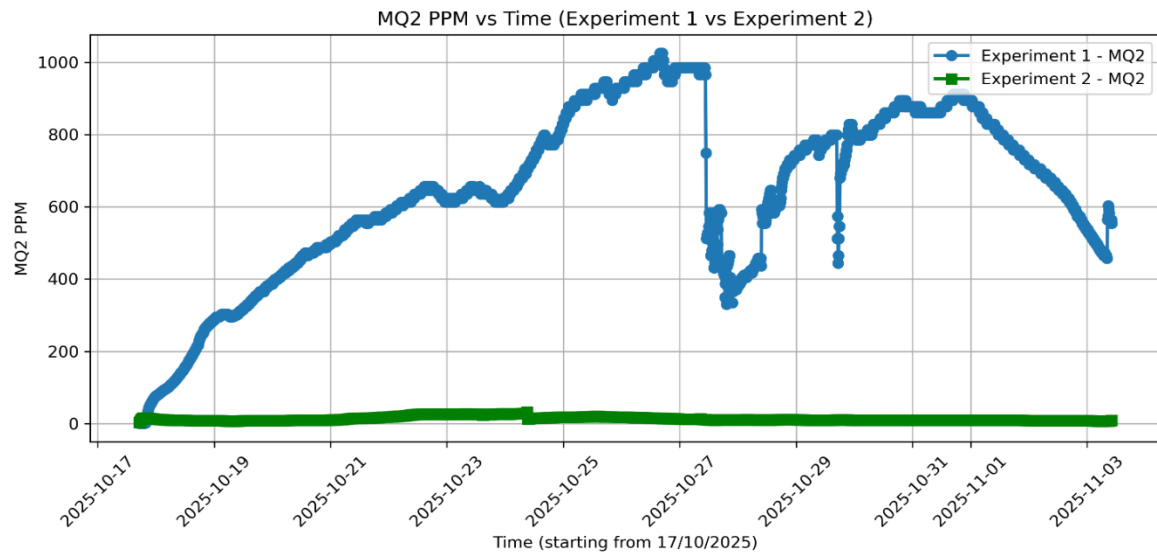
- **Curves used:**

$MQ-2 \approx 50 \cdot (Rs/R0)^{-2.3}$ (hydrocarbons/Ethylene); $MQ-4 \approx 1000 \cdot (Rs/R0)^{-2.95}$ (Methane).

- **Reading the graph:** More gas $\rightarrow$ lower $Rs/R0 \rightarrow$ higher ppm. **Peaks** = gas bursts; **flat/low** = clean or stable air.

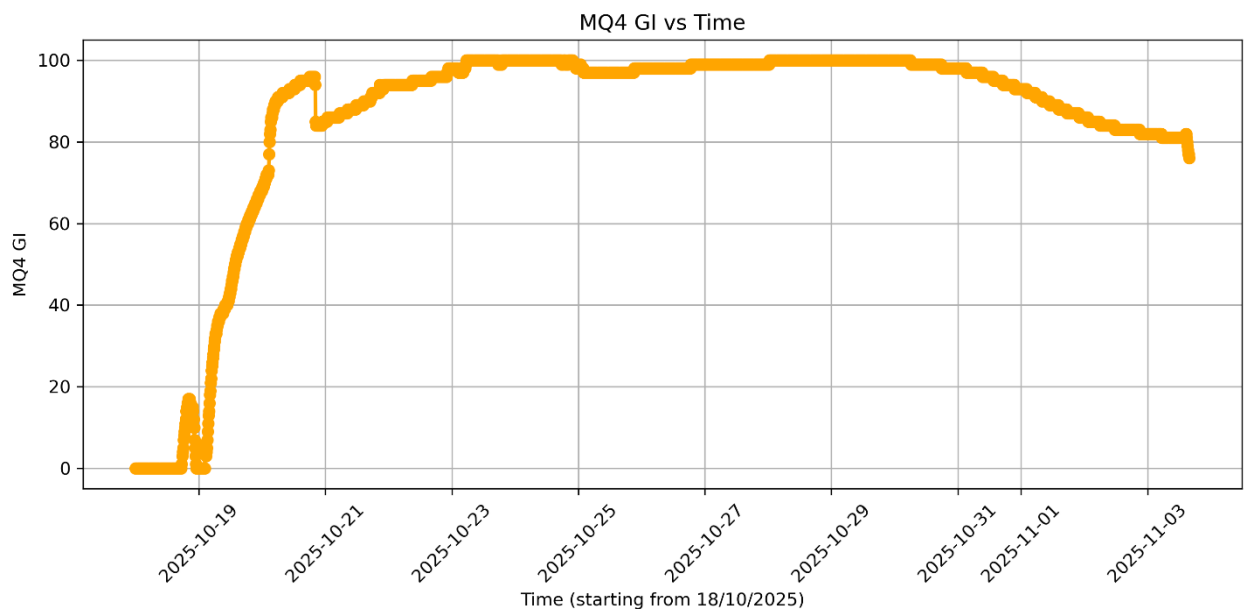
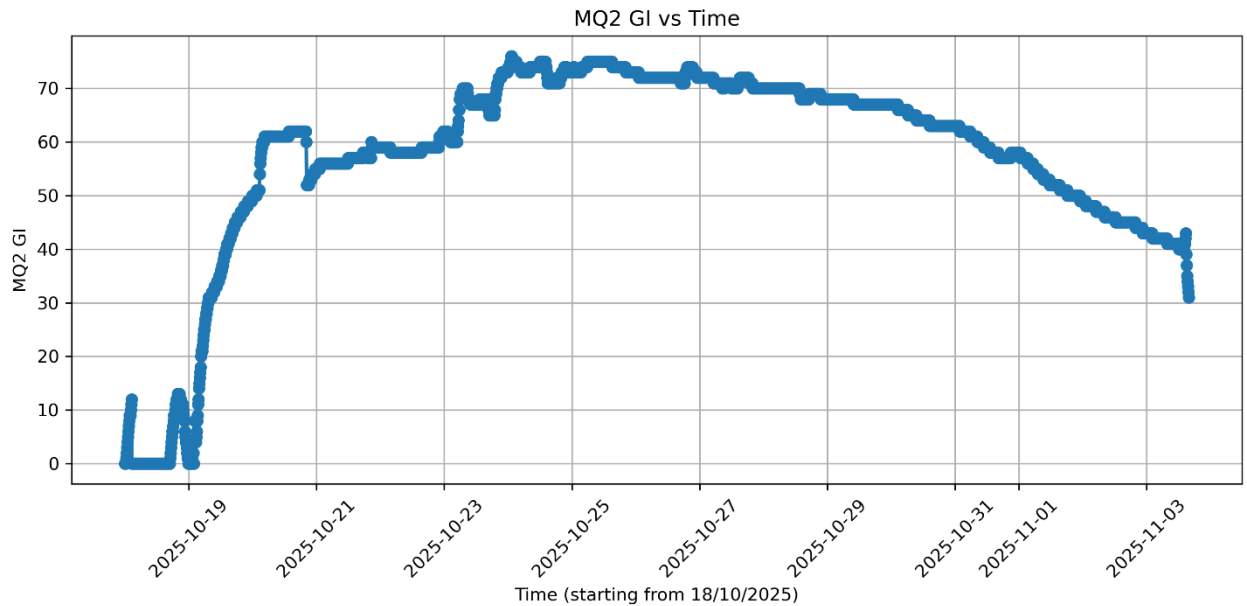


Experiment 1 & 2 Combine:



Experiment 3: Combined Gas Effect (Gas Index 0–100)

- **Y-axis** (Gas Index 0–100): Higher index = more target gas.
- **Pipeline:** $\text{ADC} \rightarrow \text{Vout} \rightarrow \text{Rs} = \text{RL} \cdot (5/\text{Vout} - 1) \rightarrow (\text{Rs}/\text{R0}) \rightarrow \text{GasIndex}$.
- **Terms:** Rs = sensor resistance (drops with more gas); R0 = clean-air baseline (from calibration); RL = fixed load resistor.
- **Meaning:** $\text{Rs}/\text{R0}$ captures overall gas effect (mixture). GasIndex is a normalized, human-friendly scale (not gas-specific ppm).
- **Normalization mapping:**
 - $\text{Rs}/\text{R0} \geq 1.0 \rightarrow \text{GI} = 0$ (fresh air)
 - $\text{Rs}/\text{R0} \approx 0.6 \rightarrow \text{GI} \approx 40$
 - $\text{Rs}/\text{R0} \approx 0.3 \rightarrow \text{GI} \approx 80$
 - $\text{Rs}/\text{R0} \leq 0.1 \rightarrow \text{GI} = 100$



5. Conclusion

This project successfully demonstrated a **Smart Fruit Decay Monitoring System** capable of detecting and analyzing gas emissions during the ripening and spoilage of fruits. Using **MQ-2 and MQ-4 sensors**, the system effectively tracked variations in **ethylene and methane** concentrations, which directly correlate with fruit freshness. The first BLE-based phase enabled **real-time wireless monitoring and data logging**, while the second NFC-based phase introduced a **low-power, offline alternative** that made the setup more practical for long-term experiments and field use.

Through two experimental rounds with banana samples, consistent gas trends were observed, **ethylene rising early during ripening, methane peaking later with decay**, and a clear **Gas Index increase** representing the combined gas effect. These results confirmed that gas sensors can reliably indicate fruit freshness and decay stages.

Overall, the project achieved its objectives by integrating **sensor technology, embedded systems, and wireless communication** into one compact, low-cost solution. In the future, the system can be enhanced by combining both BLE and NFC for hybrid operation, adding **cloud connectivity, AI-based spoilage prediction**, and **LoRa** for long-range data transfer, transforming it into a scalable, intelligent fruit monitoring platform.